Beyond the Chat Window: User-Mediated Collaboration Across AI Platforms

This manuscript (permalink) was automatically generated from TaylorResearchLab/beyond-the-chat-window@46c8f6c on August 13, 2026.

Authors

- **Deanne M. Taylor** ✉
  · taylordm
  Department of Biomedical and Health Informatics, Children's Hospital of Philadelphia, Philadelphia, Pennsylvania, USA; Department of Pediatrics, Perelman School of Medicine, University of Pennsylvania, Philadelphia, Pennsylvania, USA

✉ — Correspondence possible via GitHub Issues or email to Deanne M. Taylor <>.

Abstract

Large language models are commonly used in isolated chat sessions even when the underlying work extends across weeks, repositories, protected computing environments, and multiple model providers. I describe a **human-governed, User-mediated** workflow in which a User coordinates GPT and Claude through a structured Notion workspace. The User initiates cross-agent transitions, sometimes through detailed scientific instructions and sometimes through a minimal synchronization prompt such as “Check the log.” A Collaboration Log, numbered notebooks, canonical-state pages, and state-anchoring handoffs externalize project memory. GitHub retains versioned code and Manubot manuscript source, while the User controls protected execution, acceptance, merge, and release. Across a retrospective set of private project workspaces, both platforms produced accepted contributions in lead and review roles. The User judged neither platform consistently superior; their strengths were complementary and their roles were reversible. The approach improved traceability and practical support for FAIR research relative to chat-only work, but it was not self-driving. The User remained scientific lead, collaborator, project manager, mediator, and curator of the record. This manuscript is itself being developed through the same Notion, GitHub, and Manubot workflow as a live demonstration of feasibility.

Introduction

Multi-agent language-model systems often place several agents inside one orchestration framework, assign specialized roles, and automate their exchanges [1,2,3]. My practical problem was different. I wanted to work concurrently with two separately hosted commercial platforms on sustained scientific and technical projects without placing either model inside the other model’s runtime. The platforms had distinct context windows, tools, sandboxes, and access boundaries. They could not depend on hidden shared memory or a continuously running conversation.

The resulting method is **human-governed and User-mediated**, rather than an autonomous human-in-the-loop system. There is no self-running agent cycle that pauses occasionally for approval. The User chooses which session should act next, what shared state it should inspect, and when scientific judgment, evidence, or redirection is required. The agents communicate indirectly through durable artifacts that the User also governs.

This report is a descriptive methods note, not a benchmark. It documents the architecture, recurrent observations, practical failure modes, and a manuscript-authoring demonstration. Project-specific scientific and creative content remains private.

A User-mediated shared-record workflow

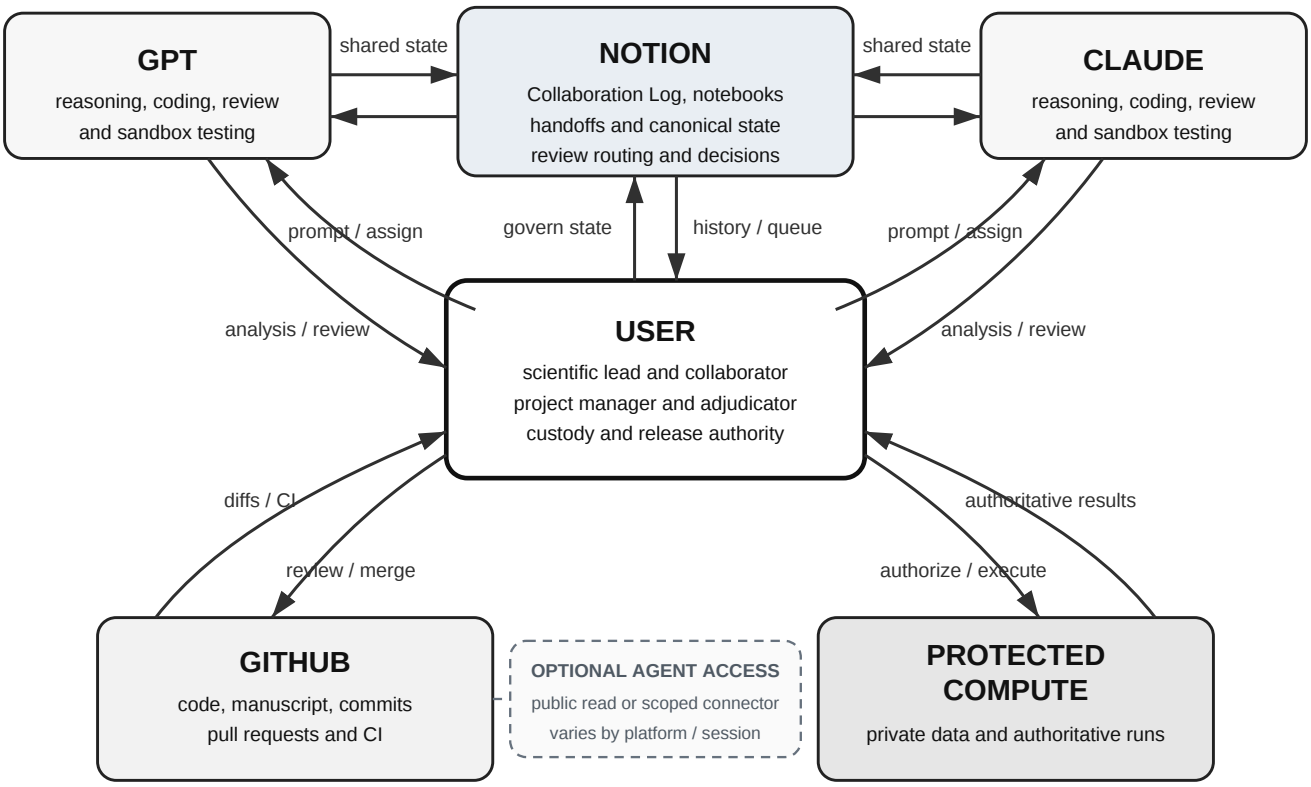

Figure 1: The User mediates the workflow. GPT and Claude read and write shared state in Notion, review versioned artifacts in GitHub, and test code in separate sandboxes. The User initiates attention shifts, controls protected execution, and accepts or rejects project state.

The work contract

The workflow begins with a concise work contract. It defines:

1. the User's authority over scope, trajectory, privacy, protected execution, acceptance, and public release;
2. durable destinations for decisions, notebooks, code, data, and manuscript source;
3. named source identities for the User and each agent session;
4. reversible lead and review roles;
5. the evidence hierarchy for model statements, sandbox runs, protected runs, and commits;
6. which actions must be entered in the Collaboration Log;
7. when a handoff is required.

Without this contract, useful work can remain trapped in chat, review flags can become stale, and several notebooks or branches can appear current at the same time.

The Collaboration Log and notebooks

The Collaboration Log is a Notion database rather than an exported transcript. Entries are typed as decisions, reviews, implementations, run results, handoffs, open questions, or memos. Common fields identify source, owner, status, assignment, reviewer requested, next action, and human approval. Code-intensive variants also record repository, branch or pull request, commit SHA, commit date, command, environment, and output location.

Numbered notebooks preserve cumulative derivations, analyses, design states, or literature audits. Canonical-state pages identify which branch governs. Incorrect or obsolete entries are normally marked **Superseded** rather than deleted. The negative history is therefore inspectable.

A handoff is a project-state anchor, not merely a summary. It records the current accepted state, completed and failed work, canonical notebooks and commits, unresolved questions, superseded branches, known uncertainty, and one explicit next action. The receiving session must reconcile the handoff against the log, repository, and notebooks before continuing.

Prompted mediation

During intensive exchanges, the User may alternate messages between agents using no more than:

> Check the log.

This small prompt acts as a synchronization signal. It tells the receiving session that another participant changed the durable project state and that the next task is encoded in the log, handoff, or linked artifact. The brevity of the prompt does not imply autonomy. The User still selects the recipient, monitors the exchange, maintains workflow fields, and intervenes when interpretation, evidence, or project direction requires it.

Artifact and execution custody

Notion and GitHub solve different custody problems. Notion holds rationale, decisions, reviews, handoffs, and explanatory records. GitHub holds executable code, tests, schemas, diffs, and manuscript source. Protected systems hold private data and authoritative execution when direct agent access is unavailable or inappropriate.

Both agents can run bounded tests in their own sandboxes. A sandbox success is evidence only for its recorded code and environment. It does not replace a protected rerun. A Git commit establishes artifact identity, not scientific validity. The User decides whether the combined evidence satisfies an acceptance gate.

Observations across project workspaces

The retrospective evidence base contains standardized Collaboration Logs together with additional descriptive workspaces whose specialized schemas are not pooled into the quantitative denominator. This distinction avoids treating unlike records as mechanically comparable while preserving evidence that the same collaboration pattern can appear in curation, software, scientific analysis, theory, and long-form narrative development.

Several observations recurred.

The platforms were complementary rather than hierarchical. GPT sometimes led derivation, coding, or manuscript implementation while Claude reviewed. In other phases, Claude led analysis, synthesis, coding, or drafting while GPT reproduced, reviewed, or repaired the work. The User judged both platforms to have performed well and did not observe a consistent global superiority.

Role reversal mattered. A second model added little when it merely polished or endorsed the first model’s answer. It was more useful when assigned to derive, execute, falsify, or review an artifact

from a distinct perspective. Roles could reverse on the next task.

Disagreement often proceeded through reciprocal revision. One platform raised an objection, the User directed the other to the new log entry, and the platforms exchanged concessions, retractions, and amendments until a three-way understanding emerged. Sometimes one platform adopted the other's position rather than meeting halfway. The User adjudicated when evidence underdetermined the answer or when project authority was required.

Collaborative conduct reduced correction friction. The agents generally acknowledged one another's useful contributions, addressed disagreements politely, admitted errors, and proposed repairs. Encouragement was not evidence of correctness, but it made correction and transfer of the lead role easier.

Externalized state reduced context loss. A new session could reconstruct current state from accepted entries, numbered notebooks, handoffs, and exact commits rather than from an increasingly compressed chat summary.

The User remained the project shepherd. The log did not maintain itself. The User monitored whether substantive work was recorded, maintained checkboxes and review queues, selected canonical state, requested missing handoffs, and shaped the project trajectory. The workflow redistributed labor but did not transfer responsibility.

The architecture does not inherently require different providers. Multiple named sessions from one provider could use the same schema and handoff rules. I have used related arrangements, but did not retain a comparable corpus, so this remains a plausible extension rather than an evaluated finding.

FAIR-supporting traceability and limitations

The active record provided substantially stronger practical support for FAIR research than isolated chat windows [4]. Typed entries and stable links improved findability. Managed permissions and persistent pages improved accessibility across sessions. A common schema and commit identifiers improved interoperability among platforms and tools. Recorded rationale, commands, inputs, outputs, limitations, reviews, and supersession improved reuse.

This does not mean a private Notion workspace is automatically or completely FAIR. Proprietary interfaces, private permissions, incomplete exports, stale metadata, and unlogged chat actions can limit long-term access and reuse. Archival release may still require structured exports, repository documentation, licensing, and deposition.

Notion also introduced operational friction. In a controlled test, a renamed database retained its old connector-visible title. Dense notebook pages containing child databases, equations, code, and attachments were difficult to edit safely through the API. Template duplication could carry old field names and categories into new projects. Pages accidentally created outside the intended database could lose structured properties when moved. Append-only updates, bounded pages, explicit identifiers, read-back verification, and versioned follow-on notebooks were generally safer than broad in-place rewriting.

The record is also incomplete unless the User actively maintains it. Review checkboxes are mutable state rather than append-only events, so completed reviews can disappear from a later sampling frame. Conversation can move faster than logging, leaving substantive actions temporarily outside the durable record. Multiple agents can amplify the same error, and agreement after exposure is not independent replication. Human mediation and adjudication therefore remain load-bearing.

Manuscript writing as a live demonstration

Collaborative paper writing exposed a specific platform limitation. Both agents could read the Notion record, but neither had a dependable ability to revise an existing shared Google Doc in place. The User would otherwise have to transfer accepted changes manually.

Manubot applies version control, continuous integration, automated references, and reviewable pull requests to scholarly writing [5]. For this paper, Notion remains the decision and collaboration record, while Manubot source in GitHub becomes the manuscript source of truth after import. GPT prepares or revises manuscript source on a branch. Claude can review the public repository and propose changes through a branch, pull request, or logged patch, depending on its connected permissions. The User reviews diffs, adjudicates wording, and controls merge and submission. GitHub Actions builds the formatted manuscript and exposes build failures.

This manuscript is therefore being produced with the method it describes. That reflexive use is a demonstration of feasibility, not evidence that the method improves outcomes. A public, duplicable Collaboration Log template with synthetic examples and GitHub custody columns is planned as a companion resource.

Conclusion

A persistent shared record can connect otherwise separate AI platforms into a practical scientific collaboration. The key design is not autonomy. It is human governance and active User mediation under an explicit work contract. The User initiates attention shifts, maintains canonical state, controls evidence and execution, and decides what becomes accepted. Within that structure, complementary agents can alternate among reasoning, coding, review, execution, repair, and manuscript editing while leaving a traceable record.

Future work should compare matched single-agent and cross-agent tasks using prespecified measures of defect detection, reproducibility, substantive revision, time to accepted artifact, and User review burden.

Acknowledgments

The author thanks the Manubot developers for providing an open collaborative manuscript workflow. GPT and Claude were used as scientific, coding, review, and writing tools under human supervision. They are not authors and had no authority over submission or publication.

Author contributions

Deanne M. Taylor: Conceptualization, Methodology, Investigation, Project administration, Supervision, Validation, Writing - original draft, Writing - review and editing.

AI-assistance disclosure

GPT and Claude contributed analyses, code review, manuscript drafting, and revision suggestions. All material was directed, reviewed, adjudicated, and accepted by the human author. The human author is responsible for the manuscript.

Data and code availability

The manuscript source, build configuration, and revision history will be available in the public GitHub repository associated with this paper. A public, duplicable Notion Collaboration Log template with synthetic records will be linked from the repository and final manuscript.

References

1. **AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation**
   Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, ... Chi Wang
   arXiv (2023-10-05) https://arxiv.org/abs/2308.08155

2. **MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework**
   Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, ... JÃ¼rgen Schmidhuber
   arXiv (2024-11-04) https://arxiv.org/abs/2308.00352

3. **ChatDev: Communicative Agents for Software Development**
   Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, ... Maosong Sun
   arXiv (2024-06-06) https://arxiv.org/abs/2307.07924

4. **The FAIR Guiding Principles for scientific data management and stewardship**
   Mark D Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, Gabrielle Appleton, Myles Axton, Arie Baak, Niklas Blomberg, Jan-Willem Boiten, Luiz Bonino da Silva Santos, Philip E Bourne, ... Barend Mons
   Scientific Data (2016-03-15) https://doi.org/bdd4
   DOI: 10.1038/sdata.2016.18 · PMID: 26978244 · PMCID: PMC4792175

5. **Open collaborative writing with Manubot**
   Daniel S Himmelstein, Vincent Rubinetti, David R Slochower, Dongbo Hu, Venkat S Malladi, Casey S Greene, Anthony Gitter
   PLOS Computational Biology (2019-06-24) https://doi.org/c7np
   DOI: 10.1371/journal.pcbi.1007128 · PMID: 31233491 · PMCID: PMC6611653